Java Data Structures — Cheat Sheet (Solved problems)

Java Data Structures — Cheat Sheet (Solved problems)
---

1) Array - Two Sum (HashMap) Problem: Given nums[] and target, return indices of two numbers summing to target. Solution idea: single-pass HashMap storing value->index.

Java: int[] twoSum(int[] nums, int target) { Map<Integer,Integer> m = new HashMap<>(); for (int i=0;i<nums.length;i++){ int need = target - nums[i]; if (m.containsKey(need)) return new int[]{m.get(need), i}; m.put(nums[i], i); } throw new IllegalArgumentException("No solution"); }

2) Reverse Linked List (Iterative) Problem: Reverse singly linked list.

Java: ListNode reverse(ListNode head) { ListNode prev = null; ListNode cur = head; while (cur != null) { ListNode next = cur.next; cur.next = prev; prev = cur; cur = next; } return prev; }

3) Detect Cycle in Linked List (Floyd) boolean hasCycle(ListNode head) { if (head == null) return false; ListNode slow = head, fast = head.next; while (fast != null && fast.next != null) { if (slow == fast) return true; slow = slow.next; fast = fast.next.next; } return false; }

4) Stack - Evaluate Reverse Polish Notation Use stack and parse tokens.

Java: int evalRPN(String[] tokens) { Deque<Integer> st = new ArrayDeque<>(); for (String t: tokens) { switch(t) { case "+" -> st.push(st.pop() + st.pop()); case "-" -> { int b=st.pop(), a=st.pop(); st.push(a-b); } case "*" -> st.push(st.pop() * st.pop()); case "/" -> { int b=st.pop(), a=st.pop(); st.push(a/b); } default -> st.push(Integer.parseInt(t)); } } return st.pop(); }

5) Queue - BFS on Graph (Shortest path in unweighted graph) Use Queue for level-order.

Java: int bfs(int start, int target, List<List<Integer>> adj) { int n = adj.size(); boolean[] vis = new boolean[n]; Queue<Integer> q = new ArrayDeque<>(); q.add(start); vis[start]=true; int level=0; while (!q.isEmpty()) { int sz=q.size(); for (int i=0;i<sz;i++) { int u=q.poll(); if (u==target) return level; for (int v: adj.get(u)) if (!vis[v]) { vis[v]=true; q.add(v); } } level++; } return -1; }

6) Binary Tree - Inorder Traversal (Recursive + Iterative) Recursive: void inorder(TreeNode node, List<Integer> res) { if (node==null) return; inorder(node.left,res); res.add(node.val); inorder(node.right,res); }

Iterative: List<Integer> inorderIter(TreeNode root) { List<Integer> res = new ArrayList<>(); Deque<TreeNode> st = new ArrayDeque<>(); TreeNode cur = root; while (cur != null || !st.isEmpty()) { while (cur != null) { st.push(cur); cur = cur.left; } cur = st.pop(); res.add(cur.val); cur = cur.right; } return res; }

7) Binary Search Tree - Insert & Search Insert: TreeNode insert(TreeNode root, int val) { if (root==null) return new TreeNode(val); if (val < root.val) root.left = insert(root.left, val); else root.right = insert(root.right, val); return root; }

Search: boolean search(TreeNode root, int val) { while (root != null) { if (root.val == val) return true; root = val < root.val ? root.left : root.right; } return false; }

8) Heap - Kth largest (Min-heap) int findKthLargest(int[] nums, int k) { PriorityQueue<Integer> pq = new PriorityQueue<>(); for (int n: nums) { pq.offer(n); if (pq.size() > k) pq.poll(); } return pq.peek(); }

9) Union-Find (Disjoint Set) class DSU { int[] parent, rank; DSU(int n){ parent=new int[n]; rank=new int[n]; for(int i=0;i<n;i++) parent[i]=i;} int find(int x){ return parent[x]==x?x:parent[x]=find(parent[x]);} void union(int a,int b){ a=find(a); b=find(b); if(a==b) return; if(rank[a]<rank[b]) parent[a]=b; else if(rank[b]<rank[a])

parent[b]=a; else { parent[b]=a; rank[a]++; } } }

10) Graph - DFS (recursive) void dfs(int u, boolean[] vis, List<List<Integer>> adj) { vis[u]=true; for(int v: adj.get(u)) if(!vis[v]) dfs(v,vis,adj); }

11) Topological Sort (Kahn's) List<Integer> topo(int n, List<List<Integer>> adj) { int[] indeg = new int[n]; for (int u=0;u<n;u++) for (int v: adj.get(u)) indeg[v]++; Queue<Integer> q = new ArrayDeque<>(); for (int i=0;i<n;i++) if (indeg[i]==0) q.add(i); List<Integer> order = new ArrayList<>(); while (!q.isEmpty()) { int u=q.poll(); order.add(u); for (int v: adj.get(u)) if (--indeg[v]==0) q.add(v); } return order.size()==n ? order : Collections.emptyList(); }

12) Two pointers - Container with most water int maxArea(int[] h) { int l=0, r=h.length-1, ans=0; while (l<r) { ans = Math.max(ans, Math.min(h[l],h[r])*(r-l)); if (h[l] < h[r]) l++; else r--; } return ans; }

13) Sliding Window - Longest substring without repeating chars int lengthOfLongestSubstring(String s) { int[] last = new int[256]; Arrays.fill(last, -1); int res=0, start=0; for (int i=0;i<s.length();i++) { start = Math.max(start, last[s.charAt(i)] + 1); res = Math.max(res, i - start + 1); last[s.charAt(i)] = i; } return res; }

14) Dynamic Programming - Fibonacci (bottom-up) int fib(int n) { if (n<2) return n; int a=0,b=1; for (int i=2;i<=n;i++) { int c=a+b; a=b; b=c; } return b; }

---------- Notes: - Use appropriate imports: java.util.*; define ListNode/TreeNode structures. - For interview: explain time & space complexity briefly.